Estrutura de Dados e Organização de Arquivos
Prof. David Alves
21/10/2026

Grafos
Os grafos são uma das estruturas de dados mais versáteis e poderosas da Computação. Eles modelam conexões e relacionamentos do mundo real — desde rotas em um GPS e conexões em redes sociais até dependências de compilação de código e redes de computadores.

1. O que é um Grafo?
Formalmente, um grafo G é um par ordenado $G = (V, E)$, onde:

- V **(Vértices / Nós):** Representam as entidades ou pontos do problema (ex: cidades, pessoas, páginas web).
- E **(Edges / Arestas):** Representam as conexões ou relacionamentos entre os vértices (ex: estradas, amizades, links hipertexto).

2. Tipos de Grafos
Dependendo de como as conexões se comportam, os grafos podem ser classificados em várias categorias:

- **Não Direcionados (Undirected):** As arestas não têm direção. Se o vértice A está conectado a B, B está conectado a A (ex: amizade no Facebook).
- **Direcionados (Directed ou DiGraphs):** As arestas possuem uma direção (uma via de mão única). Representado por setas (ex: seguidores no Instagram ou dependências de pacotes).
- **Ponderados (Weighted):** Cada aresta possui um "peso" ou custo associado (ex: a distância em quilômetros entre duas cidades ou o custo de latência de uma rede).
- **Cíclicos vs. Acíclicos:** Um grafo cíclico contém pelo menos um ciclo (caminho que começa e termina no mesmo vértice). Grafos direcionados acíclicos (DAGs) são muito usados para ordenação de tarefas (como git commit trees ou compilação).

3. Representação Computacional
Na programação (seja em C ou Python), precisamos de uma forma eficiente de armazenar grafos na memória. As duas abordagens principais são:

A. Matriz de Adjacência
Uma matriz bidimensional de tamanho |V| x |V|, onde a célula $M[i][j]$ indica se há uma aresta entre o vértice i e o vértice j (e o peso dela, se ponderado).

- **Vantagens:** Consulta rápida para saber se existe aresta entre dois nós ($O(1)$).
-
- **Desvantagens:** Consome muita memória ($O(|V|^2)$), especialmente em **grafos esparsos** (poucas conexões).

B. Lista de Adjacência
Cada vértice armazena uma lista (ou dicionário/vetor) contendo todos os seus vizinhos diretos.

- **Vantagens:** Muito eficiente em termos de memória para grafos esparsos ($O(|V| + |E|)$).

- **Desvantagens:** Verificar a existência de uma aresta específica exige percorrer a lista de adjacências de um nó.

4. Algoritmos de Busca (Travessia)
Como percorremos todos os nós de um grafo de forma sistemática? Os dois algoritmos fundamentais são:

Busca em Largura (BFS - Breadth-First Search)
- **Como funciona:** Explora o grafo por "camadas" ou níveis. Visita todos os vizinhos diretos de um nó antes de avançar para os vizinhos dos vizinhos.
-
- **Estrutura de dados:** Utiliza uma **Fila (Queue)** (FIFO - First In, First First Out).
-
- **Aplicação clássica:** Encontrar o **caminho mais curto** em um grafo não ponderado.
-

Busca em Profundidade (DFS - Depth-First Search)
- **Como funciona:** Vai o mais fundo possível em um ramo antes de retroceder (*backtracking*).

- **Estrutura de dados:** Utiliza uma **Pilha (Stack)** (LIFO - Last In, First Out), seja explicitamente ou via recursão.

- **Aplicação clássica:** Detecção de ciclos, verificação de conectividade e ordenação topológica.

5. Aplicações Práticas Avançadas
- **Caminho Mínimo (Algoritmo de Dijkstra / Bellman-Ford):** Essencial em sistemas de navegação (Waze, Google Maps) para encontrar a rota mais rápida entre dois pontos considerando pesos nas vias.

- **Árvore Geradora Mínima (Algoritmos de Prim / Kruskal):** Usada para conectar todos os vértices com o menor custo total de arestas possível (ex: cabeamento de redes elétricas ou de fibra óptica).

- **Redes de Fluxo (Maximum Flow):** Otimização de escoamento de água, tráfego ou dados através de gargalos de rede.

1. Matriz de Adjacência

Imagine uma tabela onde as linhas e colunas representam os vértices. Se o grafo tem 5 vértices, temos uma matriz 5×5.

- Se há uma aresta entre o vértice 0 e o vértice 2, a célula M[0][2] recebe 1 (ou o peso da aresta).
- **Vantagem:** Saber se dois nós estão conectados é instantâneo ($O(1)$), o que é ótimo para algoritmos que consultam a existência de arestas o tempo todo.
- **Desvantagem:** Desperdício severo de memória se o grafo for **esparso** (o que é a maioria na vida real). Se você tiver 10.000 nós, mas cada um se conectar a apenas 2 outros, a matriz terá 100 milhões de posições, sendo que 99,8% delas serão zero.

2. Lista de Adjacência

Cada vértice possui sua própria "agenda" de contatos (uma lista encadeada ou vetor dinâmico) guardando apenas os nós com os quais ele faz fronteira diretamente.

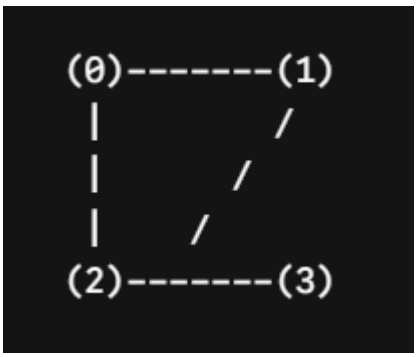
- **Vantagem:** Extremamente eficiente em termos de memória. Ocupa apenas o espaço necessário para armazenar as conexões reais que existem ($O(V + E)$).
- **Desvantagem:** Para saber se existe uma aresta entre u e v, você não tem acesso direto; precisa percorrer a lista de vizinhos do vértice u.

Aqui está uma representação visual clara de como as duas estruturas organizam o mesmo grafo na memória:

O Grafo Exemplo

Imagine um grafo **não direcionado** com **4 vértices** (0, 1, 2, 3) e as seguintes conexões (arestas):

- **0** conecta com **1** e **2**
- **1** conecta com **0** e **2**
- **2** conecta com **0**, **1** e **3**
- **3** conecta com **2**



1. Matriz de Adjacência

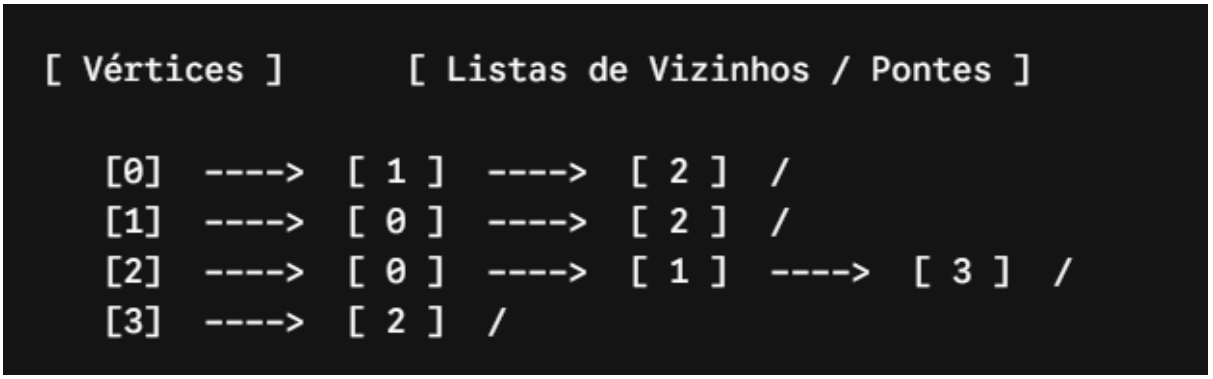
A matriz mapeia todas as combinações de linhas (origem) e colunas (destino). O valor **1** indica existência de conexão e **0** a ausência.



O que observar: Como o grafo é **não direcionado**, a matriz é **simétrica** em relação à diagonal principal (o triângulo superior é um espelho do inferior).

2. Lista de Adjacência

Aqui temos um array fixo de tamanho 4 (para os vértices 0 a 3). Cada posição aponta para uma lista encadeada contendo **apenas** os seus vizinhos diretos:



O que observar: Não há desperdício de espaço com zeros ou valores nulos. Cada elemento guarda apenas a referência para quem realmente está conectado a ele.